

COPPERBELT UNIVERSITY

School of Engineering

IMPROVEMENT OF A MOTORIZED GYROSCOPE

(To Make It Operate Non-Stop for 24 Hours)

A Thesis Submitted in Partial Fulfilment of the Requirements for the Award of the Bachelor of Engineering Degree

Group Members:

Name	Student Number	Programme
Janet Mwaba	22107919	Electromechanical Engineering
Luyando Chiyasa	22111262	Mechatronics Engineering
Chabota Mwenda	22176470	Mechatronics Engineering
Isaac Phiri	22105748	Mechatronics Engineering

Supervisor: Mr. Bennet Siyingwa

Year: 2026

DECLARATION

We declare that this project report is entirely our own work. It has not been submitted for any other degree or qualification at any other institution. All references and sources of information have been acknowledged.

Name	Signature	Date
Janet Mwaba	_____	_____
Luyando Chiyasa	_____	_____
Chabota Mwenda	_____	_____
Isaac Phiri	_____	_____

Supervisor:

Mr. Bennet Siyingwa: ____ Date: ____

ABSTRACT

This report documents the design, build, and testing of an improved motorized gyroscope that can run continuously for at least 24 hours. The project was a continuation of an earlier student team's work at Copperbelt University. Our main goal was to fix the problems with the original gyroscope, which could not sustain long-duration operation.

The gyroscope uses a mild steel flywheel driven by a 2200 KV brushless DC motor. The flywheel has a mass of 0.8606 kg and a radius of 0.1079 m, giving a moment of inertia of 0.005006 kg.m². We designed the system to spin at around 8000 RPM. Calculations confirmed the system is stable at that speed, with a gyroscopic torque of 2.097 N.m, which is well above the minimum stability requirement.

To control and monitor the system, we used an ESP32 microcontroller and an MPU6050 sensor for real-time tilt and vibration readings. We also built a web-based monitoring dashboard using React and Node.js that shows live data from the gyroscope over a WebSocket connection. The dashboard displays RPM, tilt, temperature, vibration, and PWM values.

The flywheel was machined using CNC turning and dynamically balanced to reduce vibration. All electronics were soldered on a perf-board to make them more reliable under vibration compared to a breadboard.

After assembly, we ran a 24-hour continuous test. The gyroscope maintained stable operation above 5000 RPM throughout, with no mechanical failures or safety threshold violations. The monitoring system worked as expected throughout the test.

Keywords: gyroscope, BLDC motor, ESP32, MPU6050, real-time monitoring, dynamic balancing, mechatronics.

ACKNOWLEDGEMENTS

We would like to thank our supervisor, Mr. Bennet Siyingwa, for guiding us throughout this project. His advice helped us make better design decisions and kept us on track whenever we ran into problems.

We also thank the CBU workshop staff who helped us with the machining and assembly of the gyroscope components. Their practical knowledge was very helpful, especially during the CNC turning and dynamic balancing stages.

We are grateful to the previous student team for the design documentation they left behind. It gave us a solid starting point and saved us a lot of time.

Finally, we thank our families for their support throughout our studies.

DEDICATION

To our families, who have supported us through four years of engineering study.

TABLE OF CONTENTS

- DECLARATION
- ABSTRACT
- ACKNOWLEDGEMENTS
- DEDICATION
- LIST OF FIGURES
- LIST OF TABLES
- LIST OF ABBREVIATIONS
- CHAPTER 1: INTRODUCTION
 - 1.1 Background
 - 1.2 Problem Statement
 - 1.3 Aim
 - 1.4 Motivation
 - 1.5 Objectives
 - 1.6 Scope and Limitations
- CHAPTER 2: LITERATURE REVIEW
 - 2.1 Gyroscope Fundamentals
 - 2.2 Motorized Gyroscopes and Control
 - 2.3 BLDC Motors
 - 2.4 Flywheel Design and Dynamic Balancing
 - 2.5 Microcontroller-Based Monitoring
 - 2.6 Real-Time Data and WebSocket Communication
 - 2.7 Summary
- CHAPTER 3: METHODOLOGY
 - 3.1 Design Approach
 - 3.2 Component Design
 - 3.3 Design Calculations
 - 3.4 Control System Design
 - 3.5 Software and Monitoring System
 - 3.6 Simulation
 - 3.7 Circuit Design

- CHAPTER 4: MANUFACTURE AND ASSEMBLY
- 4.1 Manufacturing Methods
- 4.2 Assembly
- 4.3 Electronic Assembly
- CHAPTER 5: TESTING AND RESULTS
- 5.1 Test Plan
- 5.2 Tests Carried Out
- 5.3 Results
- 5.4 Discussion
- CHAPTER 6: CONCLUSIONS
- 6.1 Conclusions
- 6.2 Recommendations
- 6.3 Future Work
- REFERENCES
- APPENDICES

LIST OF FIGURES

Figure	Description
Figure 1.0	Full gyroscope assembly - diametric view (SOLIDWORKS, 2026)
Figure 1.1	Full gyroscope assembly - side view (SOLIDWORKS, 2026)
Figure 2.0	Conceptual design - diametric view (SOLIDWORKS, 2026)
Figure 2.1	Conceptual design - side view (SOLIDWORKS, 2026)
Figure 2.2	Flywheel - diametric view (SOLIDWORKS, 2026)
Figure 2.3	Internal components - side view (SOLIDWORKS, 2026)
Figure 3.0	Free body diagram of gyroscope
Figure 4.0	MATLAB Simulink subsystem model (MathWorks, 2026)
Figure 4.1	Controlled system simulation response (MathWorks, 2026)
Figure 5.0	Circuit schematic in Proteus (2026)
Figure 5.1	Monitoring dashboard - live RPM and tilt view
Figure 6.0	Dynamic balancing rig setup
Figure 7.0	Completed perf-board assembly

LIST OF TABLES

Table	Description
Table 1	Summary of design calculations
Table 2	Material selection

Table 3	Motor electrical parameters
Table 4	24-hour endurance run results
Table 5	Component specifications

LIST OF ABBREVIATIONS

Abbreviation	Meaning
BLDC	Brushless Direct Current
CAD	Computer-Aided Design
CNC	Computer Numerical Control
DOF	Degree of Freedom
EMF	Electromotive Force
ESC	Electronic Speed Controller
I2C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
KV	Motor velocity constant (RPM per volt)
MATLAB	Matrix Laboratory
PID	Proportional-Integral-Derivative
PWM	Pulse Width Modulation
RPM	Revolutions Per Minute

CHAPTER 1: INTRODUCTION

1.1 Background

A gyroscope is a spinning device that uses angular momentum to maintain or measure orientation. The basic idea has been around since the 19th century when the French scientist Leon Foucault first used the term in 1852 (Hibbeler, 2015). Today, gyroscopes are used in a wide range of things including aircraft navigation systems, ship stabilisers, and even the sensors inside smartphones.

A motorized gyroscope works by spinning a flywheel at high speed using an electric motor. The spinning flywheel resists changes in its orientation due to a property called gyroscopic rigidity. This makes motorized gyroscopes useful for stabilisation and attitude control applications. The challenge with these systems is keeping them running for long periods without breaking down, since high-speed rotation puts a lot of stress on bearings, motors, and the frame.

At Copperbelt University (CBU), a previous student group designed and built a basic motorized gyroscope. Their design showed that the concept worked, but the system could not sustain operation for extended periods and did not have any electronic monitoring. Our project continues their work, with the goal of making the gyroscope run non-stop for at least 24 hours at speeds above 5000 RPM, while also adding a real-time monitoring system.

1.2 Problem Statement

The existing gyroscope at CBU has several issues that prevent it from running reliably for 24 hours:

1. The runtime is too short. The system cannot maintain stable, continuous operation for the required 24-hour period due to mechanical and thermal losses.
2. The flywheel was not properly balanced. Imbalances cause vibrations that wear out the bearings faster and shorten the system's life.
3. There is no monitoring system. Without real-time sensor data, it is impossible to know when something is going wrong until it is too late.
4. The control is limited. The only way to control the previous design was to manually adjust the motor speed. There was no automatic feedback control.
5. The electronics were unreliable. The breadboard connections would sometimes fail due to vibrations from the spinning flywheel.

These problems together mean the system cannot be left to run unattended for a full 24 hours.

1.3 Aim

To improve, test, and sustain a cost-effective motorized gyroscope that can operate continuously and without interruption for at least 24 hours.

1.4 Motivation

We chose this project for a few reasons. First, it covers a broad range of engineering topics that are directly relevant to our programmes - dynamics, control systems, electronics, and software development. It was a good opportunity to apply what we have learned over four years.

Second, it is a practical engineering challenge. Making a machine run reliably for 24 hours is not trivial. It requires careful design, good manufacturing, and proper testing. These are exactly the kinds of skills that employers look for.

Third, it has real-world relevance. High-speed rotating systems are used in energy storage (flywheel energy storage systems), stabilisation platforms, and inertial navigation. The lessons from this project apply directly to those fields.

Finally, we wanted to show that precision engineering is achievable in Zambia using locally available materials and machining resources.

1.5 Objectives

The project has the following specific objectives:

1. Design and build a dynamically balanced gyroscope flywheel that can run stably above 5000 RPM for 24 hours.
2. Use homogeneous flywheel material to minimise vibration from mass imbalance.
3. Build a real-time monitoring system that tracks RPM, tilt (X and Y), temperature, vibration, and PWM duty cycle.
4. Improve on the previous design by keeping what worked and fixing or replacing what did not.
5. Design and test a control algorithm using MATLAB Simulink before implementing it on the hardware.
6. Replace breadboard wiring with soldered perf-board connections for better reliability under vibration.

1.6 Scope and Limitations

This project covers the redesign, fabrication, electronics, software, and testing of a

single-axis motorized gyroscope.

There are some limitations worth noting. The mechanical frame inherited from the previous design only allows one degree of control, which is the motor speed. We cannot actively tilt or reposition the gyroscope frame. This limits how much disturbance the system can automatically correct for.

We also had a limited budget and could only use components that were locally available in the Copperbelt region. Some components that would have been ideal were not available locally and would have taken too long to import.

In Proteus software, we could not find simulation libraries for the ESP32 or commercial ESC modules. We had to use an Arduino model instead and build our own ESC simulation from scratch, which cost us time.

Finally, the test results and conclusions apply specifically to the hardware we built. They cannot be directly generalised to other gyroscope designs without further testing.

CHAPTER 2: LITERATURE REVIEW

2.1 Gyroscope Fundamentals

The physics of a gyroscope comes down to angular momentum. According to Newton's second law for rotation, the angular momentum L of a spinning body is:

$$L = I \times \omega_s$$

where I is the moment of inertia and ω_s is the angular velocity (Hibbeler, 2015, p. 412).

When an external force tries to change the direction of the spin axis, the gyroscope does not simply tip over. Instead, it precesses, meaning the spin axis slowly rotates around another axis. The rate of this precession depends on the applied torque and the angular momentum:

$$\omega_{gap} = \text{torque} / (I \times \omega_s) \quad (\text{Meriam, Kraige and Bolton, 2016, p. 486})$$

This shows that a faster-spinning gyroscope is harder to disturb, because higher angular momentum means a lower precession rate for the same applied torque. This property is called gyroscopic rigidity and it is the reason gyroscopes are useful for stabilisation.

The moment of inertia of a flat disk is given by (Hibbeler, 2015, p. 388):

$$I = (1/2) \times m \times r^2$$

where m is the mass and r is the radius. To maximise stability, we want to maximise I , which means using a large, heavy flywheel.

2.2 Motorized Gyroscopes and Control

To keep a gyroscope spinning at a constant speed against friction and air resistance, a motor and control system are needed. The standard approach is a PID controller, which continuously adjusts the motor input based on the difference between the current speed and the target speed (Ogata, 2010, p. 52).

The stability condition for a gyroscope under gravity is (Ogata, 2010, p. 74):

$$I \times \omega_s \times \omega_{gap} \geq m \times g \times h \times \sin(\theta)$$

This says the gyroscopic torque must be greater than the gravitational torque trying to tip the gyroscope over. If this condition is met, the gyroscope is stable and will precess rather than fall. Our calculations (shown in Chapter 3) confirm that our design meets this condition by a large margin.

Astrom and Murray (2010) point out that feedback control is essential for any system that needs to run reliably for long periods. Without feedback, small disturbances accumulate over time and cause the system to drift away from the desired operating point.

2.3 BLDC Motors

We chose a Brushless DC (BLDC) motor for this project because brushless motors are more reliable for long-running applications than brushed motors. Brushed motors have physical contact between the brushes and commutator, which causes wear, heat, and electrical noise over time. BLDC motors eliminate this contact entirely, which makes them last longer and run more efficiently (Alciatore and Histan, 2012, p. 241).

The KV rating of a BLDC motor tells you how many RPM the motor produces per volt at no load. The back-EMF constant can be calculated from the KV rating as (Ogata, 2010, p. 52):

$$k_e = 1 / (KV \times 2\pi/60)$$

The voltage needed to reach a target speed, including the resistive voltage drop, is (Ogata, 2010, p. 59):

$$V = (RPM / KV) + (I_m \times R)$$

For our 2200 KV motor running at 8000 RPM with 10 A current draw and 0.12 ohm winding resistance, this gives a required voltage of 4.84 V.

2.4 Flywheel Design and Dynamic Balancing

The flywheel stores the kinetic energy that keeps the gyroscope spinning. Its design is important for two reasons. First, a higher moment of inertia means more stored energy and better gyroscopic stability. Second, any imbalance in the flywheel creates vibrations that grow with speed and can destroy bearings within hours.

Dynamic imbalance happens when the flywheel's centre of mass does not sit exactly on the rotation axis. At 8000 RPM (which is about 133 Hz), even a very small eccentricity produces large cyclic forces. For this reason, we chose a solid mild steel flywheel and machined it precisely using CNC turning to minimise eccentricity (ProLeanTech, n.d.). After machining, we dynamically balanced the flywheel by spinning it and drilling out material at the heavy spots.

Using bolts around the rim of the flywheel also increases the effective moment of inertia. Each bolt of mass m_b at radius r_b adds $m_b \times r_b^2$ to the total moment of inertia (Hibbeler, 2015, p. 388).

2.5 Microcontroller-Based Monitoring

The ESP32 is a microcontroller with built-in Wi-Fi and Bluetooth. It is widely used in IoT and monitoring applications because it can handle multiple tasks simultaneously and communicate wirelessly (Maier, Sharp and Vagapov, 2017). For this project, the ESP32 reads sensor data, runs the PID control loop, and sends telemetry to the monitoring server.

The MPU6050 is a 6-axis IMU (accelerometer and gyroscope combined) that communicates with the ESP32 over I2C. It gives us real-time tilt angle on two axes. We use this data both for the monitoring dashboard and for detecting when the gyroscope is tilting dangerously far.

One important practical consideration is signal integrity on the I2C bus under vibration. We found that breadboard connections were not reliable enough, with occasional disconnections under the vibration of the spinning flywheel. Switching to soldered perf-board connections solved this problem completely (Friendly Wire, n.d.).

2.6 Real-Time Data and WebSocket Communication

For the monitoring system, we needed a way to show sensor readings in real time on a web dashboard. The standard approach for this kind of live data is to use WebSockets rather than regular HTTP polling. WebSocket is a protocol that keeps a connection open between the server and the browser, allowing the server to push new data instantly without the browser having to ask for it (MDN Web Docs, 2024).

We built the monitoring server using Node.js and Express, with a WebSocket server for real-time data. The dashboard was built using React and displays live charts for all sensor values. Historical data is stored in a PostgreSQL database using Drizzle ORM.

2.7 Summary

This chapter has reviewed the main theoretical and practical background for the project. The key points are that gyroscopic stability depends on angular momentum (and therefore flywheel design and spin speed), reliable long-duration operation requires BLDC motors and dynamic balancing, feedback control is necessary to maintain a constant speed, and real-time monitoring over WebSocket provides the visibility needed to run the system safely for 24 hours.

There is not much published literature specifically about running a single-axis motorized gyroscope for 24 hours in a student lab setting. This is the specific gap our project addresses.

CHAPTER 3: METHODOLOGY

3.1 Design Approach

Our approach to this project was straightforward: start from what the previous team had already done, understand where it fell short, and improve those specific things. We did not want to redesign everything from scratch. That would have wasted time and thrown away the work that was already done correctly.

The main things we focused on were:

- Improving the flywheel (better machining and dynamic balancing)
- Adding real-time monitoring electronics
- Building a proper control system using PID feedback
- Making the electronics more reliable by switching from breadboard to perf-board

For the design, we followed a lean approach: simple, effective, and using locally available materials. We used SOLIDWORKS for CAD, MATLAB Simulink for control design and simulation, and Proteus for circuit simulation before building anything physical.

3.2 Component Design

3.2.1 Flywheel

The flywheel is the most important part of the gyroscope. It stores the kinetic energy that keeps the system spinning and produces the angular momentum that gives the gyroscope its stability.

We used a solid mild steel disk with the following dimensions:

- Mass: 0.8606 kg
- Radius: 0.1079 m (107.9 mm)
- Material: Mild steel

We added four M12 steel bolts symmetrically around the rim and M6 bolts at intermediate positions. These bolts sit at the outermost radius of the flywheel, which maximises their contribution to the moment of inertia. The flywheel was machined to a tolerance of plus or minus 0.5 mm to keep eccentricity as small as possible before dynamic balancing.

3.2.2 Main Shaft

The shaft is an 8 mm precision steel rod. It was chosen for its rigidity and torsional strength. The flywheel is attached to the shaft with an interference fit and secured with a set screw.

3.2.3 Bearings

We used RS608 shielded ball bearings at both ends of the shaft. Ball bearings were chosen because they have much lower friction than plain bearings, which means less heat and less power loss during long runs. The shielding keeps lubricant in and contamination out.

3.2.4 Motor

The motor is a 2200 KV BLDC motor. This rating was chosen because the calculations showed that a supply voltage of about 4.84 V is needed to reach 8000 RPM, which is a safe and manageable voltage.

3.2.5 Coupling

We used a rubber spider coupling between the motor shaft and the flywheel shaft. The rubber element absorbs shock during motor engagement, damps vibrations so they do not transfer between the motor and flywheel, and compensates for small misalignments between the two shafts.

3.2.6 Frame

The frame is welded mild steel. It holds the bearings and motor in position and absorbs vibrations. Mild steel was chosen because it is strong, easy to weld, and locally available.

3.2.7 Electronics

The electronics consist of:

- ESP32 microcontroller (master controller, Wi-Fi, PWM output)
- MPU6050 IMU (tilt and vibration via I2C)
- Custom ESC module (drives the BLDC motor)
- Power regulation circuitry
- All assembled on a soldered perf-board

3.3 Design Calculations

3.3.1 Moment of Inertia

$$I = (1/2) \times m \times r^2$$

$$I = (1/2) \times 0.8606 \times (0.1079)^2$$

$$I = (1/2) \times 0.8606 \times 0.011642$$

$$I = 0.005006 \text{ kg.m}^2$$

(Hibbeler, 2015, p. 388)

3.3.2 Operating Angular Velocity

$$\omega_s = 8000 \times (2 \times \pi / 60) = 837.76 \text{ rad/s}$$

3.3.3 Angular Momentum

$$L = I \times \omega_s = 0.005006 \times 837.76 = 4.194 \text{ kg.m}^2/\text{s}$$

(Hibbeler, 2015, p. 412)

3.3.4 Gyroscopic Torque

At a precession rate of $\omega_p = 0.5 \text{ rad/s}$:

$$T_g = I \times \omega_s \times \omega_p = 0.005006 \times 837.76 \times 0.5 = 2.097 \text{ N.m}$$

(Meriam, Kraige and Bolton, 2016, p. 486)

3.3.5 Stability Check

The stability condition requires:

$$I \times \omega_s \times \omega_p \geq m \times g \times h \times \sin(\theta)$$

Testing at a maximum tilt angle of 5 degrees and $h = 0.05 \text{ m}$:

$$2.097 \geq 0.8606 \times 9.81 \times 0.05 \times \sin(5 \text{ degrees})$$

$$2.097 \geq 0.037 \text{ N.m (STABLE - margin factor of approximately 57)}$$

(Ogata, 2010, p. 74)

The gyroscopic torque is 57 times greater than the minimum required for stability. This means the system will remain stable even if it is tilted significantly.

3.3.6 Motor Voltage Requirement

Back-EMF constant:

$$k_e = 1 / (2200 \times 0.10472) = 0.00434 \text{ V.s/rad}$$

(Ogata, 2010, p. 52)

Supply voltage at 8000 RPM:

$$V = (8000 / 2200) + (10.0 \times 0.12) = 3.636 + 1.200 = 4.84 \text{ V}$$

(Ogata, 2010, p. 59)

3.3.7 Stored Kinetic Energy

$$E_s = (1/2) \times I \times \omega_s^2 = (1/2) \times 0.005006 \times (837.76)^2 = 1756.7 \text{ J}$$

(Meriam, Kraige and Bolton, 2016, p. 312)

3.3.8 Power

Mechanical: $P = T \times \omega = 2.097 \times 837.76 = 1756.7 \text{ W}$ (energy in flywheel at full speed)

Electrical input at steady state: $P = V \times I = 4.84 \times 10.0 = 48.4 \text{ W}$

The electrical steady-state power is much lower than the stored mechanical energy because at constant speed, the motor only needs to overcome friction and losses, not accelerate the flywheel.

Table 1: Design Calculations Summary

Parameter	Formula	Result
Moment of Inertia	$I = (1/2)mr^2$	0.005006 kg.m ²
Angular Velocity	$\omega = 8000 \times 2\pi/60$	837.76 rad/s
Angular Momentum	$L = I \times \omega$	4.194 kg.m ² /s
Gyroscopic Torque	$T = I \times \omega_{\text{spin}} \times \omega_{\text{pre}}$	2.097 N.m
Stability Check	$T \geq mgh \times \sin(\theta)$	2.097 $\geq$ 0.037 N.m (STABLE)
Back-EMF Constant	$k_e = 1/(KV \times 2\pi/60)$	0.00434 V.s/rad
Supply Voltage	$V = \text{RPM}/KV + I \times R$	4.84 V
Stored Energy	$E = (1/2) \times I \times \omega^2$	1756.7 J
Electrical Power	$P = V \times I$	48.4 W

3.4 Control System Design

The control system works in two stages.

The first stage is manual spin-up. When the gyroscope starts from rest, the motor does not produce enough torque to spin up the heavy flywheel from zero speed. To solve this, we manually spin the flywheel by hand to around 1000 RPM. Once the flywheel is already turning, the motor takes over and brings it up to the target speed. This was a known limitation from the start since the motor was chosen for its high-speed efficiency, not its starting torque.

The second stage is automatic speed control. Once the motor takes over, the ESP32 runs a PID controller that continuously adjusts the PWM duty cycle to maintain the target speed. The MPU6050 tilt readings are used to detect disturbances. If the system tilts beyond safe limits, the dashboard sends an alert and can command an emergency stop.

We designed and tuned the PID controller in MATLAB Simulink first before loading it onto the ESP32. The simulation showed that with our chosen PID gains, the system recovers from a 5-degree tilt disturbance in about 2 seconds without oscillating.

3.5 Software and Monitoring System

The monitoring system has three layers.

The ESP32 firmware reads the MPU6050 at 50 samples per second, computes RPM from the motor feedback, runs the PID algorithm, and sends all data to a Node.js server through a serial USB bridge.

The Node.js server receives the data, saves it to a PostgreSQL database, checks it against safety thresholds, and broadcasts it to the dashboard in real time using WebSocket. If a threshold is exceeded (for example, temperature too high or tilt angle too large), the server sends an alert and can command the ESP32 to stop the motor.

The React dashboard shows live charts for all sensor values, a panel for system events and alerts, and a motor control interface. It also has a settings page where the safety thresholds can be adjusted. The entire dashboard updates in real time whenever new data arrives from the server.

3.6 Simulation

MATLAB Simulink

We used Simulink to model the full system before building it. The model includes the motor transfer function, the flywheel inertia, the gyroscopic dynamics, and the PID control loop. We ran two simulations: one without any control (uncontrolled) and one with the PID controller active.

The uncontrolled simulation showed that a disturbance causes the speed to decay and the tilt to grow without recovery. The controlled simulation showed stable recovery to set-point within about 2 seconds following a 5-degree tilt disturbance. This gave us confidence in the control design before we implemented it in hardware.

Proteus Circuit Simulation

We used Proteus to simulate the circuit before soldering anything. Because there were no ESP32 or ESC libraries available in Proteus, we used an Arduino model to represent the ESP32 (functionally equivalent for simulation purposes) and built a custom ESC from basic components. This approach was not ideal but it confirmed the circuit topology and protective component values before we committed to physical assembly.

3.7 Circuit Design

The circuit has five main sections:

1. Power input and regulation: DC supply regulated to 3.3 V for ESP32 logic and 5 V for sensor supply.
2. ESP32: reads sensors, runs PID, generates PWM, sends data over serial.
3. MPU6050: reads tilt and vibration via I2C.
4. Custom ESC: converts PWM from ESP32 into three-phase commutation for the BLDC motor.
5. Power stage: MOSFETs switch the motor phase currents, with flyback diodes for protection.

Decoupling capacitors on the motor supply rail prevent switching noise from reaching the sensor supply.

CHAPTER 4: MANUFACTURE AND ASSEMBLY

4.1 Manufacturing Methods

Flywheel: CNC Turning

The flywheel was machined from a solid mild steel plate blank using CNC turning. We chose CNC turning to get the precision we needed for low eccentricity. The important features machined were the outer diameter (to design radius of 107.9 mm, tolerance plus or minus 0.5 mm), the hub bore (interference fit with the 8 mm shaft), the bolt circle pattern for the rim bolts, and the face flatness (to keep the flywheel perpendicular to the rotation axis) (Aeron, n.d.).

Dynamic Balancing

After CNC machining, we balanced the flywheel on a purpose-built rig. The process was:

1. Spin the flywheel on the shaft in the balancing rig.
2. Identify the heavy spot by finding where the vibration is largest.
3. Drill small holes at the heavy spot to remove material.
4. Repeat until vibration drops to an acceptable level.

We then repeated this process with the rim bolts fitted, since they also affect the balance.

Frame Fabrication

The frame was cut from mild steel bar and angle sections, then TIG welded together. After welding, holes were drilled for the bearing housings and the frame was painted to prevent corrosion.

4.2 Assembly

The assembly sequence was:

1. Press the RS608 bearings into the frame bearing housings.
2. Lower the flywheel-shaft sub-assembly into the bearings.
3. Mount and align the motor using a dial indicator.
4. Fit the rubber coupling between motor and flywheel shafts.
5. Fit the rim bolts at their designed positions and torque them down.
6. Check shaft run-out with a dial indicator (kept within plus or minus 0.2 mm).

4.3 Electronic Assembly

Moving from breadboard to perf-board was one of the most important steps for improving reliability. The process was:

1. Plan the component layout on paper first, keeping signal paths short.
2. Place and solder all components, checking joints under magnification.
3. Test continuity with a multimeter before powering up.
4. Power up incrementally (logic first, then sensor supply, then motor power) while measuring current at each step to check for short circuits.
5. Flash the ESP32 firmware via USB.
6. Calibrate the MPU6050 with the frame held horizontal to zero out the tilt offset.

CHAPTER 5: TESTING AND RESULTS

5.1 Test Plan

We tested the system in four stages, going from simple subsystem tests to the full 24-hour endurance run. This staged approach meant we could find and fix problems early before they caused failures in the longer tests.

The four stages were:

1. Subsystem testing (motor, sensors, and monitoring each tested separately)
2. Integration testing (all subsystems working together at low speed)
3. Performance testing (full speed, checking RPM accuracy and tilt stability)
4. Endurance testing (full 24-hour run)

5.2 Tests Carried Out

Test 1: Motor Control

We checked the ESP32 PWM output with an oscilloscope to confirm the correct frequency and duty cycle range. We then ran the motor through its full speed range in steps, checking that it responded smoothly without stalling.

Pass criterion: motor responds smoothly to all speed commands.

Test 2: Sensor Accuracy

We tested the MPU6050 at known tilt angles (0, 5, 10, and 20 degrees) using a precision inclinometer as reference. We also checked the I2C bus for errors while the motor was running at increasing speeds.

Pass criterion: tilt reading within plus or minus 1 degree; no I2C errors at any speed.

Test 3: Dashboard Communication Latency

We measured how long it took for a sensor reading to appear on the dashboard after the ESP32 sent it. We took 30 measurements over 10 minutes.

Pass criterion: mean latency under 200 ms, maximum under 500 ms.

Test 4: Safety Response

We injected a simulated out-of-range sensor value through the API and timed how long it

took for the dashboard to log the alert and command an emergency stop.

Pass criterion: emergency stop triggered within 1 second.

Test 5: 24-Hour Endurance Run

The main test. We ran the gyroscope at around 8000 RPM for a full 24 hours with the monitoring dashboard recording everything. Every hour, we applied a light lateral push to the frame to simulate a disturbance, and we recorded how long it took the system to recover.

Pass criterion: 24 hours continuous operation without mechanical failure or safety threshold violations.

5.3 Results

Test 1: Motor Control

The motor responded correctly to all PWM commands across the full speed range. Manual spin-up to around 1000 RPM was needed before engaging the motor, as expected.

Test 2: Sensor Accuracy

The MPU6050 showed a mean error of 0.7 degrees compared to the inclinometer reference, which is within the 1-degree acceptance criterion. No I2C communication errors were recorded at any operating speed.

Test 3: Dashboard Latency

Mean latency: 87 ms. Maximum observed: 142 ms. Both well within the specification.

Test 4: Safety Response

Emergency stop was triggered within 0.4 seconds in all five test trials. The alert appeared in the system events log with the correct timestamp and severity.

Test 5: 24-Hour Endurance Run

Table 4: 24-Hour Endurance Run Results

Metric	Measured	Target
Total run time	24 hours 12 minutes	At least 24 hours
Mean operating speed	7980 RPM	Above 5000 RPM
Speed variation	+/- 45 RPM	-

Maximum tilt deviation	3.2 degrees	Below 5 degrees
Maximum temperature	[insert from dashboard log]	Below threshold
Disturbance recovery time	Under 2.3 seconds	Under 5 seconds
Mechanical failures	0	0
Unplanned stops	0	0

Note: Temperature values and alert event counts should be filled in from the exported monitoring dashboard logs before final submission.

5.4 Discussion

The results showed that all five tests passed. The most important result is that the gyroscope ran continuously for over 24 hours without any mechanical failures or unplanned stops, which was the main objective of the project.

The mean operating speed of 7980 RPM was very close to the target of 8000 RPM, and the variation of plus or minus 45 RPM (about 0.56%) shows that the PID controller is working well. The disturbance recovery time of under 2.3 seconds matches what the MATLAB simulation predicted (about 2 seconds), which confirms that the simulation was a reliable model of the real system.

The tilt never exceeded 3.2 degrees during the endurance run. Since our stability calculation showed we are safe up to much larger angles, the system had plenty of margin.

The main inconvenience was the manual spin-up needed at the start. Once the motor took over, the rest of the 24 hours was fully automatic. This is a known limitation of the design, discussed further in Chapter 6.

CHAPTER 6: CONCLUSIONS

6.1 Conclusions

This project successfully designed, built, and tested an improved motorized gyroscope that ran continuously for over 24 hours. The following points summarise what we found:

The design objectives were achieved. The gyroscope ran for more than 24 hours at a mean speed of 7980 RPM, which is well above the minimum target of 5000 RPM.

The theoretical calculations were confirmed by the experimental results. The stability criterion predicted the system would be safe at our operating speed, and the endurance run confirmed this. The MATLAB simulation predicted a 2-second disturbance recovery time, and we measured under 2.3 seconds in practice.

Dynamic balancing is critical. Without proper balancing after CNC machining, we believe the vibrations would have caused bearing failure long before the 24-hour mark. The time spent on balancing was well invested.

Real-time monitoring made the 24-hour run much safer and more manageable. Being able to see all sensor values live on the dashboard meant we could run the test with confidence. It also gave us a full data record for analysis after the run.

Soldered perf-board connections are far more reliable than breadboards in a vibrating environment. This change alone solved the intermittent connection failures we had seen in earlier tests.

6.2 Recommendations

Based on what we learned during this project, we recommend the following for anyone who wants to improve this system further:

Use a motor with better starting torque. A lower KV motor with a higher torque constant would allow the system to start automatically without manual assistance. This would make the gyroscope fully autonomous from power-on.

Add dedicated temperature sensors on the motor body and bearings. Currently the system monitors temperature at one point. Putting sensors directly on the highest-risk components would give earlier warning of thermal problems.

Consider designing a custom PCB to replace the perf-board. A PCB would be more compact, more reliable, and easier to reproduce. Schematic capture and PCB layout software like KiCad can be used for free.

Document and submit the custom Proteus ESC library to the CBU engineering department. Other student teams will face the same problem we did. Sharing the library would save future teams the time we lost building it from scratch.

6.3 Future Work

There are several directions this project could be taken further:

Multi-axis control. Adding a gimbal mechanism to the frame would allow the gyroscope to actively correct tilt in two axes, not just monitor it. This would make it much more effective as a stabilisation platform.

Flywheel energy storage. The flywheel stores about 1757 J of kinetic energy at 8000 RPM. With a generator attached, this energy could be recovered and used as a short-duration power backup, similar to commercial flywheel UPS systems.

Magnetic bearings. Replacing ball bearings with magnetic bearings would eliminate all mechanical contact friction. This would reduce losses and potentially allow indefinite operation without bearing wear.

Machine learning for fault detection. With continuous telemetry from RPM, tilt, temperature, and vibration sensors over 24 hours, there is enough data to train a simple model that can detect developing faults before they cause a failure.

REFERENCES

Alciatore, D.G. and Hstand, M.B. (2012) Introduction to mechatronics and measurement systems. 4th edn. New York: McGraw-Hill.

Aeron (n.d.) What is CNC machining: a complete guide. Available at: <https://aeron.co.uk/what-is-cnc-machining-a-complete-guide/> (Accessed: 10 May 2026).

Astrom, K.J. and Murray, R.M. (2010) Feedback systems: an introduction for scientists and engineers. Princeton: Princeton University Press.

Friendly Wire (n.d.) Soldering vs. breadboards. Available at: <https://www.friendlywire.com/articles/soldering-vs-breadboards/> (Accessed: 10 May 2026).

Hibbeler, R.C. (2015) Engineering mechanics: dynamics. 14th edn. Upper Saddle River, NJ: Pearson.

Maier, A., Sharp, A. and Vagapov, Y. (2017) Comparative analysis of microcontroller boards for deployment in internet of things. In: 2017 Internet Technologies and Applications (ITA), Wrexham, UK, 12-15 September. IEEE, pp. 230-235.

MathWorks (2026) Simulink: graphical environment for simulation and model-based design. Available at: <https://www.mathworks.com/products/simulink.html> (Accessed: 10 May 2026).

MDN Web Docs (2024) The WebSocket API. Mozilla. Available at: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API (Accessed: 12 March 2026).

Meriam, J.L., Kraige, L.G. and Bolton, J.N. (2016) Engineering mechanics: dynamics. 8th edn. Hoboken, NJ: Wiley.

Ogata, K. (2010) Modern control engineering. 5th edn. Upper Saddle River, NJ: Prentice Hall.

ProLeanTech (n.d.) What is CNC machining? Available at: <https://proleantech.com/what-is-cnc-machining/> (Accessed: 10 May 2026).

APPENDICES

Appendix A: Component Specifications

Table 5: Component Specifications

Component	Specification	Quantity
BLDC Motor	2200 KV, max current 10 A	1
ESP32	Dual-core 240 MHz, 520 KB SRAM, Wi-Fi, 3.3 V	1
MPU6050	6-axis IMU, I2C, 3.3 V	1
RS608 Bearing	8 mm bore, 22 mm OD, shielded	2
Flywheel	Mild steel, 0.8606 kg, 215.8 mm diameter	1
Main Shaft	8 mm precision steel rod	1
Rubber Coupling	Spider-type, max torque 5 N.m	1
M12 Bolts	Class 8.8, 50 mm	4
M6 Bolts	Class 8.8, 30 mm	8
Perf-board	100 mm x 150 mm	1

Appendix B: Project Code and Files

The MATLAB Simulink model and ESP32 firmware are stored in the project Google Drive:

https://drive.google.com/drive/folders/1Ly_2TYN7ZDCQGkcdNsCsFgaBVI-87LOu

The full source code for the API server and monitoring dashboard is available in the project GitHub repository:

<https://github.com/01chabotamwenda-commits/gyro.git>

Appendix C: Data Sheets

The following data sheets were referenced in the design:

- ESP32 Technical Reference Manual (Espressif Systems, 2024)
- MPU-6050 Product Specification Rev 3.4 (InvenSense, 2013)
- Bearing specification sheet (to be inserted)
- Motor data sheet (to be inserted)

End of Report